

SECCIÓN: REGLAS ARQUITECTÓNICAS Y CONTEXTO OCULTO (¡LEER SIEMPRE!)

1. La Ilusión del "SaaS" (Enfoque estricto en Backend Edge-Ready): Aunque el título del proyecto mencione "SaaS", **ESTÁ EstrictAMENTE PROHIBIDO generar código para frameworks web pesados (como Django, React, Vue, etc.)**. El proyecto es en realidad un "Backend-as-a-Service" y un procesador de datos diseñado para operar en sistemas empotrados (Edge/Linux). Cualquier interfaz visual será un simple dashboard de telemetría muy ligero (ej. scripts en Python plano o un Flask muy básico) solo para monitorizar el entrenamiento y las inferencias.

2. Límites del Hardware (Validación "Software-in-the-Loop"): El sistema está diseñado para correr en una Raspberry Pi, pero **el alcance actual NO incluye hardware físico, ni conexiones IoT reales (Modbus, MQTT), ni actuadores eléctricos**. Todo el despliegue es "Software-in-the-Loop". Las pruebas de producción se ejecutan aislando el código dentro de contenedores **Docker** para simular el entorno Linux Edge. No escribas scripts para leer sensores físicos; todo se lee del dataset simulado (`dataset_final.csv`).

3. El Límite del Agente DQN (Decisiones Macro, no Micro): El agente de Reinforcement Learning (DQN) **NO gestiona casas individuales**. Su espacio de acción se limita a 13 decisiones macroscópicas sobre el **excedente total** y el **déficit total** de la comunidad (ej. cargar batería, vender a red). El reparto interno de los coeficientes de energía a los 15 vecinos no es tarea de la IA; si hay que implementarlo, debe ser un bloque de código puramente matemático y determinista que se ejecuta *después* de que la IA decida la acción macro.

4. Separación Estricta: Entrenamiento vs. Inferencia: Existe una barrera arquitectónica inquebrantable entre el entorno de desarrollo y el de producción:

- **Entrenamiento:** Usa `Stable Baselines3`, `Gymnasium`, `Pandas` y `Numpy`.
- **Producción/Inferencia (Docker/Edge):** El modelo se exporta a **ONNX**. El script del simulador final debe ser extremadamente ligero y usar únicamente `ONNX Runtime` y `Numpy` (para el `argmax()` de las 13 acciones). No importes librerías pesadas de Machine Learning en los scripts de despliegue.